INGEGNERIA DEL SOFTWARE

ESERCITAZIONE 2

Gabriele Paveri-Fontana
Mail: gabriele.paveri@polimi.it

15 Ottobre 2025

Recap Access Modifiers

Visibilità delle classi:

public	visibile a tutte le classi (con import del package)
«friendly» (package private, default)	visibile solo all'interno dello stesso package

Visibilità di attributi e metodi:

public	visibile a tutte le classi (con import del package)
protected	visibile alle classi nello stesso package e alle sottoclassi
«friendly» (package private, default)	visibile alle classi nello stesso package
private	visibile solo all'interno della classe stessa

Esercizio 1 Access Modifiers

Completare i codici seguenti con gli opportuni modificatori di visibilità in modo tale che l'accesso alle variabili e ai metodi sia il più ristretto possibile ma che non crei errori di compilazione (errori a compile-time).

Esercizio 1a Access Modifiers

Completare il codice con gli opportuni modificatori di visibilità.

```
package a;

... class First {
        ... int x;
        ... int y;
        ... void h() { y = -1; }
}

... class Second extends First {
        ... void f(int x) {
            this.x = x;
            h();
        }
}
```

```
package b;
imports a.*;

... class Third {
        public static void main(String[] args) {
            Second s = new Second();
            s.f(3);
}
```

Esercizio 1a Access Modifiers

Risposta:

```
package a;

class First {    //default
        int x;    //default
        private int y;
        void h() { y = -1; }    //default
}

public class Second extends First {
        public void f(int x) {
            this.x = x;
            h();
        }
}
```

```
package b;
imports a.*;

class Third {    //default
        public static void main(String[] args) {
            Second s = new Second();
            s.f(3);
}
```

Esercizio 1b Access Modifiers

Cosa cambia se aggiungiamo la classe *Fourth*?

```
package a;

... class First {
        ... int x;
        ... int y;
        ... void h() { y = -1; }
}

... class Second extends First {
        ... void f(int x) {
            this.x = x;
            h();
        }
}
```

```
package b;
imports a.*;

… class Third {
        public static void main(String[] args) {
            Second s = new Second();
            s.f(3);
}

… class Fourth extends First {
        void g() { h(); }
}
```

Esercizio 1b Access Modifiers

Risposta:

```
package a;

public class First {
        int x;    //default
        private int y;
        protected void h() { y = -1; }
}

public class Second extends First {
        public void f(int x) {
            this.x = x;
            h();
        }
}
```

```
package b;
imports a.*;

class Third {    //default
        public static void main(String[] args) {
            Second s = new Second();
            s.f(3);
}

class Fourth extends First {    //default
        void g() { h(); }
}
```

Esercizio 2a Access Modifiers

Stabilire se il codice seguente è corretto.
Se lo è, che output produce una chiamata a *C2.m1()*?

```
package a;
public class C1 {
        public void m1() {
            System.out.print(“C1.m1-”);
        }
        protected void m2() {
            System.out.print(“C1.m2-”);
        }
        private void m3() {
            System.out.print(“C1.m3-”);
        }
}
```

```
package b;
import a.*;
public class C2 extends C1 {
        public void m1() {
            System.out.print(“C2.m1-”);
            m2();
            m3();
        }
        protected void m2() { System.out.print(“C2.m2-”); }
        private void m3() { System.out.print(“C2.m3-”); }
}
```

Esercizio 2a Access Modifiers

Stabilire se il codice seguente è corretto.
Se lo è, che output produce una chiamata a *C2.m1()*?

Osservazioni:

- *C2.m1()* fa un override di *C1.m1()*, che è public.
- *C2.m2()* fa un override di *C1.m2()*, che è protected (*C2* è sottoclasse di *C1*).
- *C2.m3()* è la definizione di un nuovo metodo, in quanto *C2* non vede la definizione di *C1.m3()*, perché questo è un metodo private.

Risposta:

- Il codice è corretto.
- Una chiamata a *C2.m1()* stampa «C2.m1-C2.m2-C2.m3-».

Esercizio 2b Access Modifiers

Stabilire se il codice seguente è corretto.
Se lo è, che output produce una chiamata a *C3.m1()*?

```
package a;
public class C1 {
        public void m1() {
            System.out.print("C1.m1-");
        }
        protected void m2() {
            System.out.print("C1.m2-");
        }
        private void m3() {
            System.out.print("C1.m3-");
        }
}
```

```
package b;
import a.*;
public class C3 extends C1 {
        public void m1() {
            System.out.print("C3.m1-");
            m2();
            m3();
        }
        protected void m2() {
            System.out.print("C3.m2-");
        }
}
```

Esercizio 2b Access Modifiers

Stabilire se il codice seguente è corretto.
Se lo è, che output produce una chiamata a *C3.m1()*?

Osservazioni:

- *C3.m1()* fa un override di *C1.m1()*, che è public.
- *C3.m2()* fa un override di *C1.m2()*, che è protected (*C3* è sottoclasse di *C1*).
- *C3.m3()* non è definito e *C1.m3()* è private in *C1* e pertanto non è visibile in *C3.*

Risposta:

- Il codice è scorretto.
  Infatti in *C3.m1()* viene invocato il metodo *m3()* che non è né definito.
  né visibile in *C2*. Si ottiene quindi un errore a compile-time.

Esercizio 2c Access Modifiers

Stabilire se il codice seguente è corretto.
Se lo è, che output produce una chiamata a *C4.m1()*?

```
package a;
public class C1 {
        public void m1() {
            System.out.print("C1.m1-");
        }
        protected void m2() {
            System.out.print("C1.m2-");
        }
        private void m3() {
            System.out.print("C1.m3-");
        }
}
```

```
package b;
import a.*;
public class C4 extends C1 {
        public void m1() {
            System.out.print("C4.m1-");
            m2();
            m3();
        }
        private void m3() {
            System.out.print("C4.m3-");
        }
}
```

Esercizio 2c Access Modifiers

Stabilire se il codice seguente è corretto.
Se lo è, che output produce una chiamata a *C4.m1()*?

Osservazioni:

- *C4.m1()* fa un override di *C1.m1()*, che è public.
- *C4.m2()* non è definito, ma *C1.m2()* è protected in *C1* e pertanto è visibile in *C4*.
- *C4.m3()* è la definizione di un nuovo metodo, in quanto *C2* non vede la definizione di *C1.m3()*, perché questo è un metodo private.

Risposta:

- Il codice è corretto.
- Una chiamata a *C4.m1()* stampa «C4.m1-C1.m2-C4.m3-».

Esercizio 3a (Default) Constructors

Che output produce il seguente programma?
E che output produce se viene eliminata la definizione del costruttore nella classe *Father*?

```
class Father {
        Father() {
            System.out.println("Padre!");
        }
}

class Son extends Father {
        Son() {
            System.out.println("Figlio!");
        }
}
```

```
class Example {
        public static void main(String[] args) {
            Son s = new Son();
        }
}
```

Esercizio 3a (Default) Constructors

Che output produce il seguente programma?
E che output produce se viene eliminata la definizione del costruttore nella classe *Father*?

```
class Father {
        Father() {
            System.out.println("Padre!");
        }
}

class Son extends Father {
        Son() {
            System.out.println("Figlio!");
        }
}
```

```
class Example {
        public static void main(String[] args) {
            Son s = new Son();
        }
}
```

N.B.: Se la prima riga di un costruttore di una sottoclasse non è «*super(…);*», allora il compilatore inserisce automaticamente «*super()*».

Esercizio 3a (Default) Constructors

Che output produce il seguente programma?
E che output produce se viene eliminata la definizione del costruttore nella classe *Father*?

```
class Father {
        Father() {
            System.out.println("Padre!");
        }
}

class Son extends Father {
        Son() {
            System.out.println("Figlio!");
        }
}
```

```
class Example {
        public static void main(String[] args) {
            Son s = new Son();
        }
}
```

Risposta:

- Con costruttore definito: Padre! Figlio!
- Con costruttore di default: Figlio!

Esercizio 3b (Default) Constructors

Date le seguenti classi, che valore hanno gli attributi di *sub* in seguito all'istruzione *Subclass sub = new Subclass(1);*? E cosa succede eliminando il costruttore *Superclass()*?

```
class Superclass {
        int attr1;
        Superclass(int attr1) {
            this.attr1 = attr1;
        }
        Superclass() {
            this.attr1 = 0;
}
```

```
class Subclass extends Superclass {
        int attr2;
        Subclass(int attr2) {
            this.attr2 = attr2;
        }
}
```

Esercizio 3b (Default) Constructors

Date le seguenti classi, che valore hanno gli attributi di *sub* in seguito all'istruzione *Subclass sub = new Subclass(1);*? E cosa succede eliminando il costruttore *Superclass()*?

```
class Superclass {
        int attr1;
        Superclass(int attr1) {
            this.attr1 = attr1;
        }
        Superclass() {
            this.attr1 = 0;
}
```

```
class Subclass extends Superclass {
        int attr2;
        Subclass(int attr2) {
            this.attr2 = attr2;
        }
}
```

N.B.: Se la prima riga di un costruttore di una sottoclasse non è «*super(…);*», allora il compilatore inserisce automaticamente «*super()*».

Esercizio 3b (Default) Constructors

Date le seguenti classi, che valore hanno gli attributi di *sub* in seguito all'istruzione *Subclass sub = new Subclass(1);*? E cosa succede eliminando il costruttore *Superclass()*?

```
class Superclass {
        int attr1;
        Superclass(int attr1) {
            this.attr1 = attr1;
        }
        Superclass() {
            this.attr1 = 0;
}
```

```
class Subclass extends Superclass {
        int attr2;
        Subclass(int attr2) {
            this.attr2 = attr2;
        }
}
```

Risposta:

- Con costruttore *Superclass()*: attr1 = 0; attr2 = 1.
- Senza costruttore *Superclass()* si ottiene un errore a compile-time!

Esercizio 4 Polimorfismo

Vediamo ora una serie di esercizi riguardanti il polimorfismo.
Il concetto fondamentale è la differenza tra il tipo statico di una variabile ed il tipo dinamico dell'oggetto a cui essa si riferisce.

Tipo statico

- Valutato a compile-time.
- Immutabile, determinato dalla dichiarazione della variabile.

Tipo dinamico

- Valutato a run-time.
- Mutabile (tramite assegnamenti).
- E' un sottotipo del tipo statico, oppure uguale al tipo statico.

Una variabile di tipo *T* può riferirsi ad un qualsiasi oggetto il cui tipo sia *T* o un sottotipo di *T*.

Esercizio 4a *instanceof* VS *getClass()*

instanceof

- «*c instanceof C*» è un'espressione che restituisce *true* se il tipo dinamico di *c* è *C* o un suo sottotipo.

getClass()

- *Object.getclass()* è un metodo della classe *Object* che restituisce il tipo dinamico dell'oggetto su cui è chiamato.
- «*c1.getClass() == c2.getClass()*» vale *true* sse il tipo dinamico di *c1* coincide con il tipo dinamico di *c2*.

Esercizio 4a *instanceof* VS *getClass()*

Che output produce il seguente programma?

```
class Father { }
class Son extends Father { }
class Test {
        public static void main(String[] args) {
            Father f1 = new Son();
            Father f2 = new Father();
            System.out.println(f1 instanceof Father);
            System.out.println(f1.getClass() == f2.getClass());
        }
}
```

Esercizio 4a *instanceof* VS *getClass()*

Che output produce il seguente programma?

```
class Father { }
class Son extends Father { }
class Test {
        public static void main(String[] args) {
            Father f1 = new Son();
            Father f2 = new Father();
            System.out.println(f1 instanceof Father);
            System.out.println(f1.getClass() == f2.getClass());
        }
}
```

Risposta

- «true»
- «false»

Tipi statici

- *f1 : Father*
- *f2 : Father*

Tipi dinamici (al termine)

- *f1 : Son*
- *f2 : Father*

Esercizio 4b Casting e Assegnamenti

Casting "Verso l'alto rispetto al tipo dinamico"

Un casting «*(C) c*» di un oggetto *c* ad un tipo *C* è corretto sse il tipo dinamico di *c* è *C* o un suo sottotipo, ovvero sse «*c instanceof C == true*».

- Il tipo statico di «*(C) c*» è *C*.
- Il tipo dinamico di «*(C) c*» è il tipo dinamico di *c* prima del casting.
- La correttezza di un casting viene valutata a run-time.

Assegnamenti "Verso il basso rispetto al tipo statico"

Un assegnamento «*c1 = c2*» è corretto sse il tipo statico di *c2* è il tipo statico di *c1* o un suo sottotipo.

- Il tipo dinamico di *c1* dopo l'assegnamento è il tipo dinamico di *c2*.
- La correttezza di un assegnamento viene valutata a compile-time.

Esercizio 4b Casting e Assegnamenti

Dato il seguente frammento di codice, stabilire quali assegnamenti e quali casting sono corretti.

```
class Gen1 { }
class Gen2 extends Gen1 { }
class Gen3 extends Gen2 { }
```

```
class Test {
        public static void main(String[] args) {
           Gen1 g_11 = new Gen1();
           Gen1 g_12 = new Gen2();
           Gen1 g_13 = new Gen3();
           Gen2 g_22 = new Gen2();
           Gen2 g_23 = new Gen3();
           Gen3 g_33 = new Gen3();
        }
}
```

Esercizio 4b Casting e Assegnamenti

Dato il seguente frammento di codice, stabilire quali assegnamenti e quali casting sono corretti.

```
class Test {
        public static void main(String[] args) {
           Gen1 g_11 = new Gen1();
           Gen1 g_12 = new Gen2();
           Gen1 g_13 = new Gen3();
           Gen2 g_22 = new Gen2();
           Gen2 g_23 = new Gen3();
           Gen3 g_33 = new Gen3();
        }
}
```

Casting corretti

- *(Gen1) g_11*
- *(Gen1) g_12, (Gen2) g_12*
- *(Gen1) g_13, (Gen2) g_13, (Gen3) g_13*
- *(Gen1) g_22, (Gen2) g_22*
- *(Gen1) g_23, (Gen2) g_23, (Gen3) g_23*
- *(Gen1) g_33, (Gen2) g_33, (Gen3) g_33*

Esercizio 4b Casting e Assegnamenti

Dato il seguente frammento di codice, stabilire quali assegnamenti e quali casting sono corretti.

```
class Test {
        public static void main(String[] args) {
           Gen1 g_11 = new Gen1();
           Gen1 g_12 = new Gen2();
           Gen1 g_13 = new Gen3();
           Gen2 g_22 = new Gen2();
           Gen2 g_23 = new Gen3();
           Gen3 g_33 = new Gen3();
        }
}
```

Assegnamenti corretti

Senza bisogno di casting:

- *g_11 = g_11, g_12, g_13, g_22, g_23, g_33*
- *g_12 = g_11, g_12, g_13, g_22, g_23, g_33*
- *g_13 = g_11, g_12, g_13, g_22, g_23, g_33*
- *g_22 = g_22, g_23, g_33*
- *g_23 = g_22, g_23, g_33*
- *g_33 = g_33*

Esercizio 4b Casting e Assegnamenti

Dato il seguente frammento di codice, stabilire quali assegnamenti e quali casting sono corretti.

```
class Test {
        public static void main(String[] args) {
           Gen1 g_11 = new Gen1();
           Gen1 g_12 = new Gen2();
           Gen1 g_13 = new Gen3();
           Gen2 g_22 = new Gen2();
           Gen2 g_23 = new Gen3();
           Gen3 g_33 = new Gen3();
        }
}
```

Assegnamenti corretti

Possibili solo tramite casting:

- *g_22 = (Gen2) g_12*
- *g_22 = (Gen2) g_13*
- *g_22 = (Gen3) g_13*
- *g_23 = (Gen2) g_12*
- *g_23 = (Gen2) g_13*
- *g_23 = (Gen3) g_13*
- *g_33 = (Gen3) g_13*
- *g_33 = (Gen3) g_23*

Esercizio 4b Casting e Assegnamenti

NOTA BENE

- «*c2 instanceof ClasseStatica_c1 == true*» ~~⇒~~ «*c1 = c2*» corretto
- «*c2 instanceof ClasseStatica_c1 == true*» ⇒ «*c1 = (ClasseStatica_c1) c2*» corretto

Esempio

```
class Gen1 { }
class Gen2 extends Gen1 { }
class Gen3 extends Gen2 { }
```

```
class Test {
        public static void main(String[] args) {
           Gen1 g_13 = new Gen3();
           Gen2 g_22 = new Gen2();
           g_22 = g_13;              //errore a compile time
           g_22 = (Gen2) g_13;  //corretto
        }
}
```

Esercizio 4c Dynamic Binding

Chiamata metodi

A fronte di una chiamata ad un metodo «*c1.m(c2)*»:

Il legame (binding) tra il metodo invocato ed il metodo attivato è dinamico!

- a compile-time → Viene scelto il metodo da chiamare in base ai tipi statici di *c1* e *c2*, ovvero viene individuato il <u>prototipo</u> *C1.m(C2)*. → risoluzione overloading
- a run-time → Viene eseguito il metodo più vicino a quello con prototipo *C1.m(C2)*, in base al tipo dinamico di *c1* (e NON in base al tipo dinamico di *c2*). → risouzione overriding

Esercizio 4c Dynamic Binding

Che output produce il seguente programma?

```
class Father {
        int x;
        public Father(int x) { this.x = x; }
        public int m(Father f) {
                return (f.x - this.x);
        }
}
class Son extends Father {
        int y;
        public Son(int x, int y) {
                super(x);
                this.y = y;
        }
        public int m(Father f) { return 100; }
        public int m(Son s) {
                return super.m(s) + (s.y - this.y);
        }
}
```

```
public class Main {
        public static void main(String args[]) {
                Father f1, f2;
                Son s1, s2;
                f1 = new Father(3);
                f2 = new Son(3,10);
                System.out.println(f1.m(f2));                    //1
                System.out.println(f2.m(f1));                    //2
                s1 = new Son(4,21);
                System.out.println(s1.m(f1) + s1.m(f2));   //3
                System.out.println(f1.m(s1) + f2.m(s1));   //4
                s2 = new Son(5,22);
                System.out.println(s1.m(s2));                    //5
        }
}
```

Esercizio 4c Dynamic Binding

Che output produce il seguente programma?

```
class Father {
        int x;
        public Father(int x) { this.x = x; }
        public int m(Father f) { return (f.x - this.x); }
}
class Son extends Father {
        int y;
        public Son(int x, int y) { super(x); this.y = y; }
        public int m(Father f) { return 100; }
        public int m(Son s) { return super.m(s) + (s.y - this.y); }
}
```

Osservazioni

La classe *Son* definisce:

- un metodo *Son.m(Father)*, che effettua un overriding del metodo *Father.m(Father)*;
- un overloading di *m(…)*, tramite un metodo *Son.m(Son)*.

Esercizio 4c Dynamic Binding

Che output produce il seguente programma?

```
public class Main {
        public static void main(String args[]) {
            Father f1, f2;
            Son s1, s2;
            f1 = new Father(3);
            f2 = new Son(3,10);
            System.out.println(f1.m(f2));                  //1
            System.out.println(f2.m(f1));                  //2
            s1 = new Son(4,21);
            System.out.println(s1.m(f1) + s1.m(f2));   //3
            System.out.println(f1.m(s1) + f2.m(s1));   //4
            s2 = new Son(5,22);
            System.out.println(s1.m(s2));                  //5
        }
}
```

Risposta:

- Istruzione //1

Esercizio 4c Dynamic Binding

Che output produce il seguente programma?

```
public class Main {
        public static void main(String args[]) {
            Father f1, f2;
            Son s1, s2;
            f1 = new Father(3);
            f2 = new Son(3,10);
            System.out.println(f1.m(f2));                  //1
            System.out.println(f2.m(f1));                  //2
            s1 = new Son(4,21);
            System.out.println(s1.m(f1) + s1.m(f2));   //3
            System.out.println(f1.m(s1) + f2.m(s1));   //4
            s2 = new Son(5,22);
            System.out.println(s1.m(s2));                  //5
        }
}
```

Risposta:

- Istruzione //1 stampa «0» (= *f2.x* - *f1.x*)

➢ Overloading: *f1* ed *f2* hanno tipo statico *Father*, dunque viene cercato (e trovato) un metodo di prototipo *Father.m(Father)*, nella classe *Father*.

➢ Overriding: *f1* ha tipo dinamico *Father*, dunque viene eseguito proprio il metodo *Father.m(Father)*.

Esercizio 4c Dynamic Binding

Che output produce il seguente programma?

```
public class Main {
        public static void main(String args[]) {
            Father f1, f2;
            Son s1, s2;
            f1 = new Father(3);
            f2 = new Son(3,10);
            System.out.println(f1.m(f2));                  //1
            System.out.println(f2.m(f1));                  //2
            s1 = new Son(4,21);
            System.out.println(s1.m(f1) + s1.m(f2));   //3
            System.out.println(f1.m(s1) + f2.m(s1));   //4
            s2 = new Son(5,22);
            System.out.println(s1.m(s2));                  //5
        }
}
```

Risposta:

- Istruzione //2

Esercizio 4c Dynamic Binding

Che output produce il seguente programma?

```
public class Main {
        public static void main(String args[]) {
            Father f1, f2;
            Son s1, s2;
            f1 = new Father(3);
            f2 = new Son(3,10);
            System.out.println(f1.m(f2));                  //1
            System.out.println(f2.m(f1));                  //2
            s1 = new Son(4,21);
            System.out.println(s1.m(f1) + s1.m(f2));   //3
            System.out.println(f1.m(s1) + f2.m(s1));   //4
            s2 = new Son(5,22);
            System.out.println(s1.m(s2));                  //5
        }
}
```

Risposta:

- Istruzione //2 stampa «*100*»

➢ Overloading: *f1* ed *f2* hanno tipo statico *Father*, dunque viene cercato (e trovato) un metodo di prototipo *Father.m(Father)*, nella classe *Father*.

➢ Overriding: *f2* ha tipo dinamico *Son*, dunque viene eseguito, tra i metodi che fanno un override di *Father.m(Father)*, il metodo *Son.m(Father)*.

Esercizio 4c Dynamic Binding

Che output produce il seguente programma?

```
public class Main {
        public static void main(String args[]) {
            Father f1, f2;
            Son s1, s2;
            f1 = new Father(3);
            f2 = new Son(3,10);
            System.out.println(f1.m(f2));                  //1
            System.out.println(f2.m(f1));                  //2
            s1 = new Son(4,21);
            System.out.println(s1.m(f1) + s1.m(f2));   //3
            System.out.println(f1.m(s1) + f2.m(s1));   //4
            s2 = new Son(5,22);
            System.out.println(s1.m(s2));                  //5
        }
}
```

Risposta:

- Istruzione //3

Esercizio 4c Dynamic Binding

Che output produce il seguente programma?

```
public class Main {
        public static void main(String args[]) {
            Father f1, f2;
            Son s1, s2;
            f1 = new Father(3);
            f2 = new Son(3,10);
            System.out.println(f1.m(f2));                  //1
            System.out.println(f2.m(f1));                  //2
            s1 = new Son(4,21);
            System.out.println(s1.m(f1) + s1.m(f2));   //3
            System.out.println(f1.m(s1) + f2.m(s1));   //4
            s2 = new Son(5,22);
            System.out.println(s1.m(s2));                  //5
        }
}
```

Risposta:

- Istruzione //3 stampa «*200*» (= 100+100)

➢ Overloading: *s1* ha tipo statico *Son*, mentre *f1* ed *f2* hanno *Father*, dunque le due chiamate hanno come prototipo *Son.m(Father)*, che è un metodo di *Son*.

➢ Overriding: nessun metodo fa override di *Son.m(Father)*, dunque viene eseguito questo sia da *s1.m(f1)* sia da *s1.m(f2)* (N.B.: nonostante *f2* abbia tipo dinamico *Son*, *s1.m(f2)* NON esegue *Son.m(Son)*).

Esercizio 4c Dynamic Binding

Che output produce il seguente programma?

```
public class Main {
        public static void main(String args[]) {
            Father f1, f2;
            Son s1, s2;
            f1 = new Father(3);
            f2 = new Son(3,10);
            System.out.println(f1.m(f2));                  //1
            System.out.println(f2.m(f1));                  //2
            s1 = new Son(4,21);
            System.out.println(s1.m(f1) + s1.m(f2));   //3
            System.out.println(f1.m(s1) + f2.m(s1));   //4
            s2 = new Son(5,22);
            System.out.println(s1.m(s2));                  //5
        }
}
```

Risposta:

- Istruzione //4

Esercizio 4c Dynamic Binding

Che output produce il seguente programma?

```
public class Main {
        public static void main(String args[]) {
            Father f1, f2;
            Son s1, s2;
            f1 = new Father(3);
            f2 = new Son(3,10);
            System.out.println(f1.m(f2));                  //1
            System.out.println(f2.m(f1));                  //2
            s1 = new Son(4,21);
            System.out.println(s1.m(f1) + s1.m(f2));   //3
            System.out.println(f1.m(s1) + f2.m(s1));   //4
            s2 = new Son(5,22);
            System.out.println(s1.m(s2));                  //5
        }
}
```

Risposta:

- Istruzione //4 stampa «*101*» (= 4-3+100)

➢ Overloading: *f1* ed *f2* hanno tipo statico *Father*, *s1* ed *s2* hanno *Son*, dunque le due chiamate hanno come prototipo *Father.m(Son)*. Non esistendo un tale metodo, viene scelto l'unico metodo compatibile, ovvero *Father.m(Father)*.

➢ Overriding: *f1* ha tipo dinamico *Father*, dunque alla prima chiamata viene eseguito il metodo *Father.m(Father)*; *f2* ha tipo dinamico *Son*, dunque alla seconda chiamata viene eseguito il metodo *Son.m(Father)*.

Esercizio 4c Dynamic Binding

Che output produce il seguente programma?

```
public class Main {
        public static void main(String args[]) {
            Father f1, f2;
            Son s1, s2;
            f1 = new Father(3);
            f2 = new Son(3,10);
            System.out.println(f1.m(f2));                  //1
            System.out.println(f2.m(f1));                  //2
            s1 = new Son(4,21);
            System.out.println(s1.m(f1) + s1.m(f2));   //3
            System.out.println(f1.m(s1) + f2.m(s1));   //4
            s2 = new Son(5,22);
            System.out.println(s1.m(s2));                  //5
        }
}
```

Risposta:

- Istruzione //5

Esercizio 4c Dynamic Binding

Che output produce il seguente programma?

```
public class Main {
        public static void main(String args[]) {
            Father f1, f2;
            Son s1, s2;
            f1 = new Father(3);
            f2 = new Son(3,10);
            System.out.println(f1.m(f2));                  //1
            System.out.println(f2.m(f1));                  //2
            s1 = new Son(4,21);
            System.out.println(s1.m(f1) + s1.m(f2));   //3
            System.out.println(f1.m(s1) + f2.m(s1));   //4
            s2 = new Son(5,22);
            System.out.println(s1.m(s2));                  //5
        }
}
```

Risposta:

- Istruzione //5 stampa «*2*» (= 1+22-21)

➢ Overloading: *s1* ed *s2* hanno tipo statico *Son*, dunque viene scelto il metodo di prototipo *Son.m(Son)* della classe *Son*.

➢ Overriding: nessun metodo fa override di *Son.m(Son)*, dunque viene eseguito questo.

➢ All'interno, viene chiamato *super.m(s2)*. Staticamente, viene cercato un metodo *Father.m(Son)*, che non esiste. Dunque viene eseguito l'unico metodo compatibile, ovvero *Father.m(Father)*. Quindi, *super.m(s2)* restituisce 1 (= 5 - 4).

Esercizio 4d Compile-time VS Run-time (1)

Sia dato il seguente programma. Dovete:

1) Individuare ed eliminare le istruzioni che generano errori a compile-time.
2) Individuare ed eliminare le istruzioni che generano errori a runtime.
3) Stabilire cosa produce in output il programma.
4) Determinare il tipo statico di ciascuna variabile ed il tipo dinamico al termine dell'esecuzione del main.

Esercizio 4d Compile-time VS Run-time (1)

```
class Person {
        void greet() {
                System.out.println("Arrivederci");
        }
}
class EasyPerson extends Person {
        void greet() {
                System.out.println("Ciao");
        }
}
class FormalPerson extends Person {
        void greet() {
                System.out.println("Saluti");
        }
}
class VeryFormalPerson extends FormalPerson {
        void greet() {
                System.out.println("Distinti saluti");
        }
}
```

```
class Example {
        public static void main(String[] args) {
                Person p = new Person();
                EasyPerson ep = new EasyPerson();
                FormalPerson fp = new FormalPerson();
                VeryFormalPerson vfp = new VeryFormalPerson();
                p.greet();                          //1
                ep = p;                             //2
                p = ep;                             //3
                p.greet();                          //4
                ep = fp;                            //5
                ep.greet();                         //6
                fp.greet();                         //7
                p = new FormalPerson();             //8
                p.greet();                          //9
                fp = p;                             //10
                vfp = (VeryFormalPerson) fp;        //11
                vfp.greet();                        //12
        }
}
```

Esercizio 4d Compile-time VS Run-time (1)

```
class Example {
        public static void main(String[] args) {
                Person p = new Person();
                EasyPerson ep = new EasyPerson();
                FormalPerson fp = new FormalPerson();
                VeryFormalPerson vfp = new VeryFormalPerson();
                p.greet();                          //1
                ep = p;                             //2
                p = ep;                             //3
                p.greet();                          //4
                ep = fp;                            //5
                ep.greet();                         //6
                fp.greet();                         //7
                p = new FormalPerson();             //8
                p.greet();                          //9
                fp = p;                             //10
                vfp = (VeryFormalPerson) fp; //11
                vfp.greet();                        //12
        }
}
```

Errori a compile-time e run-time

Esercizio 4d Compile-time VS Run-time (1)

```
class Example {
        public static void main(String[] args) {
                Person p = new Person();
                EasyPerson ep = new EasyPerson();
                FormalPerson fp = new FormalPerson();
                VeryFormalPerson vfp = new VeryFormalPerson();
                p.greet();                          //1
                ep = p;                             //2
                p = ep;                             //3
                p.greet();                          //4
                ep = fp;                            //5
                ep.greet();                         //6
                fp.greet();                         //7
                p = new FormalPerson();             //8
                p.greet();                          //9
                fp = p;                             //10
                vfp = (VeryFormalPerson) fp; //11
                vfp.greet();                        //12
        }
}
```

Errori a compile-time e run-time

- //2 genera un errore a compile-time (assegnamento di un sovratipo a un sottotipo).

Esercizio 4d Compile-time VS Run-time (1)

```
class Example {
        public static void main(String[] args) {
                Person p = new Person();
                EasyPerson ep = new EasyPerson();
                FormalPerson fp = new FormalPerson();
                VeryFormalPerson vfp = new VeryFormalPerson();
                p.greet();                          //1
                ep = p;                             //2
                p = ep;                             //3
                p.greet();                          //4
                ep = fp;                            //5
                ep.greet();                         //6
                fp.greet();                         //7
                p = new FormalPerson();             //8
                p.greet();                          //9
                fp = p;                             //10
                vfp = (VeryFormalPerson) fp; //11
                vfp.greet();                        //12
        }
}
```

Errori a compile-time e run-time

- //2 genera un errore a compile-time (assegnamento di un sovratipo a un sottotipo).
- //5 genera un errore a compile-time (tipi non compatibili).

Esercizio 4d Compile-time VS Run-time (1)

```
class Example {
        public static void main(String[] args) {
                Person p = new Person();
                EasyPerson ep = new EasyPerson();
                FormalPerson fp = new FormalPerson();
                VeryFormalPerson vfp = new VeryFormalPerson();
                p.greet();                          //1
                ep = p;                             //2
                p = ep;                             //3
                p.greet();                          //4
                ep = fp;                            //5
                ep.greet();                         //6
                fp.greet();                         //7
                p = new FormalPerson();             //8
                p.greet();                          //9
                fp = p;                             //10
                vfp = (VeryFormalPerson) fp; //11
                vfp.greet();                        //12
        }
}
```

Errori a compile-time e run-time

- //2 genera un errore a compile-time (assegnamento di un sovratipo a un sottotipo).
- //5 genera un errore a compile-time (tipi non compatibili).
- //10 genera un errore a compile-time (assegnamento di un sovratipo a un sottotipo).

Esercizio 4d Compile-time VS Run-time (1)

```
class Example {
        public static void main(String[] args) {
                Person p = new Person();
                EasyPerson ep = new EasyPerson();
                FormalPerson fp = new FormalPerson();
                VeryFormalPerson vfp = new VeryFormalPerson();
                p.greet();                          //1
                ep = p;                             //2
                p = ep;                             //3
                p.greet();                          //4
                ep = fp;                            //5
                ep.greet();                         //6
                fp.greet();                         //7
                p = new FormalPerson();             //8
                p.greet();                          //9
                fp = p;                             //10
                vfp = (VeryFormalPerson) fp; //11
                vfp.greet();                        //12
        }
}
```

Errori a compile-time e run-time

- //2 genera un errore a compile-time (assegnamento di un sovratipo a un sottotipo).
- //5 genera un errore a compile-time (tipi non compatibili).
- //10 genera un errore a compile-time (assegnamento di un sovratipo a un sottotipo).
- //11 genera un errore a run-time (*VFP* non è sovratipo del tipo dinamico di *fp*, che è ancora *FP*).

Esercizio 4d Compile-time VS Run-time (1)

```
class Example {
        public static void main(String[] args) {
                Person p = new Person();
                EasyPerson ep = new EasyPerson();
                FormalPerson fp = new FormalPerson();
                VeryFormalPerson vfp = new VeryFormalPerson();
                p.greet();                      //1
                p = ep;                         //3
                p.greet();                      //4
                ep.greet();                     //6
                fp.greet();                     //7
                p = new FormalPerson();         //8
                p.greet();                      //9
                vfp.greet();                    //12
        }
}
```

Output del programma

- //1 «Arrivederci»
- //4 «Ciao»
- //6 «Ciao»
- //7 «Saluti»
- //9 «Saluti»
- //12 «Distinti saluti»

Esercizio 4d Compile-time VS Run-time (1)

```
class Example {
        public static void main(String[] args) {
                Person p = new Person();
                EasyPerson ep = new EasyPerson();
                FormalPerson fp = new FormalPerson();
                VeryFormalPerson vfp = new VeryFormalPerson();
                p.greet();                      //1
                p = ep;                         //3
                p.greet();                      //4
                ep.greet();                     //6
                fp.greet();                     //7
                p = new FormalPerson();         //8
                p.greet();                      //9
                vfp.greet();                    //12
        }
}
```

Tipi statici

- *p : Person*
- *ep : EasyPerson*
- *fp : FormalPerson*
- *vfp : VeryFormalPerson*

Tipi dinamici al termine

- *p : FormalPerson*
- *ep : EasyPerson*
- *fp : FormalPerson*
- *vfp : VeryFormalPerson*

Esercizio 4e Compile-time VS Run-time (2)

Sia dato il seguente programma. Come prima, dovete:

1) Individuare ed eliminare le istruzioni che generano errori a compile-time.
2) Individuare ed eliminare le istruzioni che generano errori a runtime.
3) Stabilire cosa produce in output il programma.
4) Determinare il tipo statico di ciascuna variabile ed il tipo dinamico al termine dell'esecuzione del main.

Esercizio 4e Compile-time VS Run-time (2)

```
package a;
public class C1 {
        public void m1() { sout("C1.m1") }
        protected void m2() {sout("C1.m2") }
        private void m3() { sout("C1.m3") }
}

package b;
import a.*;
public class C2 extends C1 {
        public void m1() { sout("C2.m1") }
        protected void m2() { sout("C2.m2") }
        private void m3() { sout("C2.m3") }
}
```

```
package c;
import a.*;
import b.*;
public class C3 {
        public static void main(String[] args) {
                C1 c1;   C2 c2;   Object o;
                c1 = new C1();          //1
                c1.m1();                //2
                c2 = new C2();          //3
                c2.m2();                //4
                c1 = c2;                //5
                c1.m1();                //6
                c2 = new C1();          //7
                o = new C1();           //8
                c2 = (C2) o;            //9
                o = new C2();           //10
                c1 = (C1) o;            //11
                c1.m1();                //12
        }
}
```

Esercizio 4e Compile-time VS Run-time (2)

```
package c;
import a.*;
import b.*;
public class C3 {
        public static void main(String[] args) {
                C1 c1;   C2 c2;   Object o;
                c1 = new C1();          //1
                c1.m1();                //2
                c2 = new C2();          //3
                c2.m2();                //4
                c1 = c2;                //5
                c1.m1();                //6
                c2 = new C1();          //7
                o = new C1();           //8
                c2 = (C2) o;            //9
                o = new C2();           //10
                c1 = (C1) o;            //11
                c1.m1();   //12
        }
}
```

Errori a compile-time e run-time

Esercizio 4e Compile-time VS Run-time (2)

```
package c;
import a.*;
import b.*;
public class C3 {
        public static void main(String[] args) {
                C1 c1;   C2 c2;   Object o;
                c1 = new C1();          //1
                c1.m1();                //2
                c2 = new C2();          //3
                c2.m2();                //4
                c1 = c2;                //5
                c1.m1();                //6
                c2 = new C1();          //7
                o = new C1();           //8
                c2 = (C2) o;            //9
                o = new C2();           //10
                c1 = (C1) o;            //11
                c1.m1();   //12
        }
}
```

Errori a compile-time e run-time

- //4 genera un errore a compile-time (*C2.m2()* è protected e *C3* non è nello stesso package di *C2* e non è neanche una sottoclasse di *C2*).

Esercizio 4e Compile-time VS Run-time (2)

```
package c;
import a.*;
import b.*;
public class C3 {
        public static void main(String[] args) {
                C1 c1;   C2 c2;   Object o;
                c1 = new C1();          //1
                c1.m1();                //2
                c2 = new C2();          //3
                c2.m2();                //4
                c1 = c2;                //5
                c1.m1();                //6
                c2 = new C1();          //7
                o = new C1();           //8
                c2 = (C2) o;            //9
                o = new C2();           //10
                c1 = (C1) o;            //11
                c1.m1();   //12
        }
}
```

Errori a compile-time e run-time

- //4 genera un errore a compile-time (*C2.m2()* è protected e *C3* non è nello stesso package di *C2* e non è neanche una sottoclasse di *C2*).
- //7 genera un errore a compile-time (si assegna a *c2* un oggetto il cui tipo statico, cioè *C1*, è un sovra-tipo del tipo statico di *c2*, cioè *C2*).

Esercizio 4e Compile-time VS Run-time (2)

```
package c;
import a.*;
import b.*;
public class C3 {
        public static void main(String[] args) {
                C1 c1;   C2 c2;   Object o;
                c1 = new C1();          //1
                c1.m1();                //2
                c2 = new C2();          //3
                c2.m2();                //4
                c1 = c2;                //5
                c1.m1();                //6
                c2 = new C1();          //7
                o = new C1();           //8
                c2 = (C2) o;            //9
                o = new C2();           //10
                c1 = (C1) o;            //11
                c1.m1();   //12
        }
}
```

Errori a compile-time e run-time

- //4 genera un errore a compile-time (*C2.m2()* è protected e *C3* non è nello stesso package di *C2* e non è neanche una sottoclasse di *C2*).
- //7 genera un errore a compile-time (si assegna a *c2* un oggetto il cui tipo statico, cioè *C1*, è un sovra-tipo del tipo statico di *c2*, cioè *C2*).
- //9 genera un errore a run-time (*C2* non è sovratipo del tipo dinamico di *o*, che è *C1*).

Esercizio 4e Compile-time VS Run-time (2)

```
package c;
import a.*;
import b.*;
public class C3 {
        public static void main(String[] args) {
            C1 c1;   C2 c2;   Object o;
            c1 = new C1();      //1
            c1.m1();            //2
            c2 = new C2();      //3
            c1 = c2;            //5
            c1.m1();            //6
            o = new C1();       //8
            o = new C2();       //10
            c1 = (C1) o;        //11
            c1.m1();            //12
        }
}
```

Output del programma

- //2 «C1.m1»
- //6 «C2.m1»
- //12 «C2.m1»

Esercizio 4e Compile-time VS Run-time (2)

```
package c;
import a.*;
import b.*;
public class C3 {
        public static void main(String[] args) {
            C1 c1;   C2 c2;   Object o;
            c1 = new C1();      //1
            c1.m1(); //2
            c2 = new C2();      //3
            c1 = c2;            //5
            c1.m1();            //6
            o = new C1();       //8
            o = new C2();       //10
            c1 = (C1) o;        //11
            c1.m1();            //12
        }
}
```

Tipi statici

- *c1 : C1*
- *c2 : C2*
- *o : Object*

Tipi dinamici al termine

- *c1 : C2*
- *c2 : C2*
- *o : C2*

Esercizio 5 UML Class Diagram

Si modellino tramite un diagramma UML delle classi le entità descritte nei seguenti esercizi della prima esercitazione:

1) Fraction (esercizio 1)
2) Polygon (esercizio 2)
3) Set con ArrayList (esercizio 3b)
4) Stack (esercizio 4)

Esercizio 6 *Student*

Progettare (UML) e implementare (Java) le classi
Person, *Student* e *Grade*.

Specifiche:

- Una persona (*Person*) ha un nome, un cognome e una data di nascita (si utilizzi *java.util.Date*).
- Uno studente (*Student*) è una persona con un id e una lista di voti.
- Un voto (*Grade*) contiene punteggio e crediti.
- La classe *Student* espone due funzionalità:
  - ✓ il metodo *meanGrade()*: calcolo media pesata;
  - ✓ il metodo *canGraduate()*: controllo se è possibile che si laurei (crediti totali >= 180).